UNIVERSIDAD NACIONAL DE COLOMBIA

Programación Orientada a Objetos 2026-2S

Actividad 2

Estudiante:Juan Miguel Fernandez Gonzalez

Correo: jufernandezgo@unal.edu.co

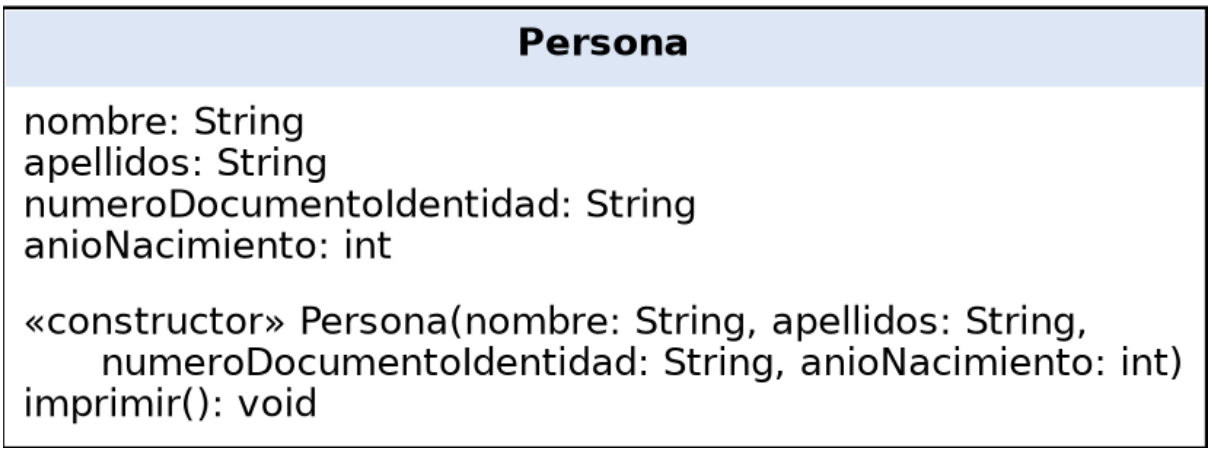
Docente: Walter Hugo Arboleda Mazo

Octubre de 2026

Ejercicio Resuelto 2.1

Se requiere un programa que modele el concepto de una persona. Una persona posee nombre, apellidos, número de documento de identidad y año de nacimiento. La clase debe tener un constructor que inicialice los valores de sus respectivos atributos.

Diagrama de clases



Código fuente (Python)

```
class Persona:

    def __init__(self, nombre: str, apellidos: str,
                 numero_documento_identidad: str, año_nacimiento: int):
        self.nombre = nombre
        self.apellidos = apellidos
        self.numero_documento_identidad = numero_documento_identidad
        self.año_nacimiento = año_nacimiento

    def imprimir(self) -> None:

        print(f"Nombre = {self.nombre}")
        print(f"Apellidos = {self.apellidos}")
        print(f"Número de documento de identidad = {self.numero_documento_identidad}")
        print(f"Año de nacimiento = {self.año_nacimiento}")
        print()

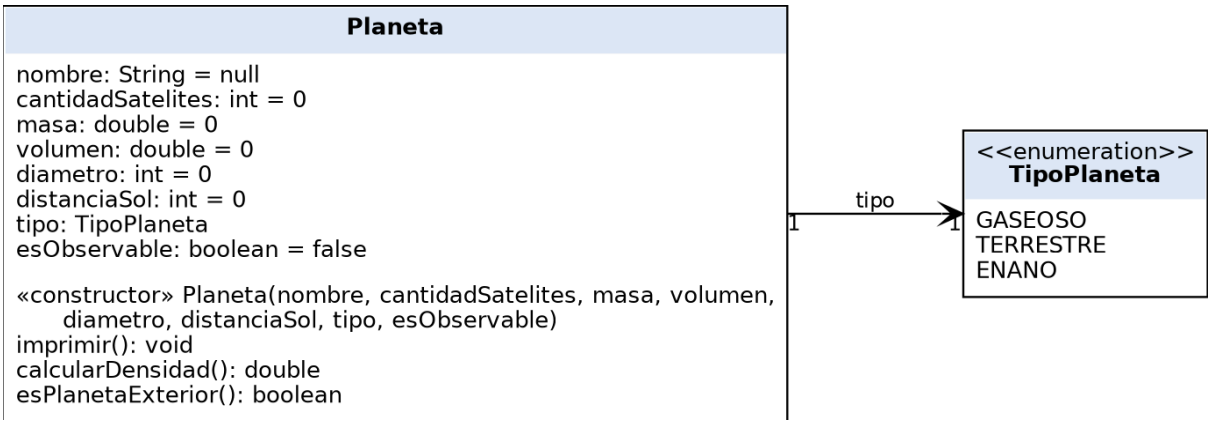

def main() -> None:
    p1 = Persona("Pedro", "Pérez", "1053121010", 1998)
    p2 = Persona("Luis", "León", "1053223344", 2001)
    p1.imprimir()
    p2.imprimir()


if __name__ == "__main__":
    main()
```

Ejercicio Resuelto 2.2

Enunciado: Se requiere un programa que modele el concepto de un planeta del sistema solar

Diagrama de clases



Código fuente (Python)

```
from enum import Enum

UA_EN_KM = 149_597_870
LIMITE_CINTURON_UA = 3.4


class TipoPlaneta(Enum):
    GASEOSO = "GASEOSO"
    TERRESTRE = "TERRESTRE"
    ENANO = "ENANO"


class Planeta:


    def __init__(self, nombre: str = None, cantidad_satelites: int = 0,
                 masa: float = 0.0, volumen: float = 0.0, diametro: int = 0,
                 distancia_sol: int = 0, tipo: TipoPlaneta = None,
                 es_observable: bool = False):
        self.nombre = nombre
        self.cantidad_satelites = cantidad_satelites
        self.masa = masa
        self.volumen = volumen
        self.diametro = diametro
        self.distancia_sol = distancia_sol
        self.tipo = tipo
        self.es_observable = es_observable

    def imprimir(self) -> None:
        print(f"Nombre del planeta = {self.nombre}")
        print(f"Cantidad de satélites = {self.cantidad_satelites}")
        print(f"Masa del planeta = {self.masa} kg")
```

```
        print(f"Volumen del planeta = {self.volumen} km³")
        print(f"Diámetro del planeta = {self.diametro} km")
        print(f"Distancia media al Sol = {self.distancia_sol} millones de km")
        print(f"Tipo de planeta = {self.tipo.value}")
        print(f"Es observable a simple vista = {self.es_observable}")

    def calcular_densidad(self) -> float:

        return self.masa / self.volumen

    def es_planeta_exterior(self) -> bool:
        """Un planeta es exterior si está más allá del cinturón de
        asteroides (3.4 UA). La distancia está en millones de km."""
        limite_millones_km = LIMITE_CINTURON_UA * UA_EN_KM / 1_000_000
        return self.distancia_sol > limite_millones_km


def main() -> None:
    p1 = Planeta("Tierra", 1, 5.9736e24, 1.08321e12, 12742, 150,
                 TipoPlaneta.TERRESTRE, True)
    p1.imprimir()
    print(f"Densidad del planeta = {p1.calcular_densidad()} kg/km³")
    print(f"Es planeta exterior = {p1.es_planeta_exterior()}")
    print()

    p2 = Planeta("Júpiter", 79, 1.899e27, 1.4313e15, 139820, 750,
                 TipoPlaneta.GASEOSO, True)
    p2.imprimir()
    print(f"Densidad del planeta = {p2.calcular_densidad()} kg/km³")
    print(f"Es planeta exterior = {p2.es_planeta_exterior()}")

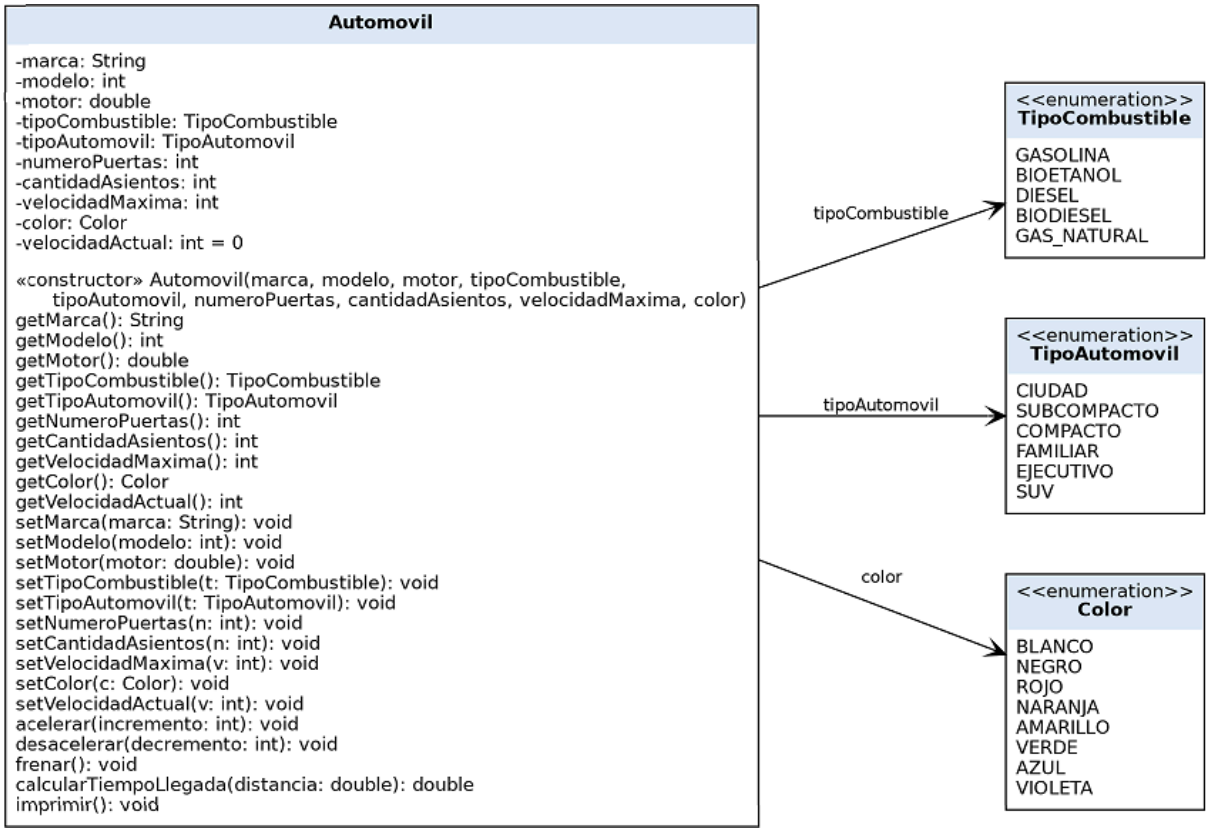

if __name__ == "__main__":
    main()
```

Ejercicio Propuesto 2.3

Se requiere un programa que modele el concepto de un automóvil. Un automóvil tiene los siguientes atributos:

- Marca: el nombre del fabricante.
- Modelo: año de fabricación.
- Motor: volumen en litros del cilindraje del motor de un automóvil.
- Tipo de combustible: valor enumerado con los posibles valores de gasolina, bioetanol, diésel, biodísel, gas natural.
- Tipo de automóvil: valor enumerado con los posibles valores de carro de ciudad, subcompacto, compacto, familiar, ejecutivo, suv.
- Número de puertas: cantidad de puertas.
- Cantidad de asientos: número de asientos disponibles que tiene el vehículo.
- Velocidad máxima: velocidad máxima sostenida por el vehículo en km/h.
- Color: valor enumerado con los posibles valores de blanco, negro, rojo, naranja, amarillo, verde, azul, violeta.
- Velocidad actual: velocidad del vehículo en un momento dado.

Diagrama de clases



Código fuente (Python)

```
from enum import Enum


class TipoCombustible(Enum):
    GASOLINA = "GASOLINA"
    BIOETANOL = "BIOETANOL"
    DIESEL = "DIESEL"
    BIODIESEL = "BIODIESEL"
    GAS_NATURAL = "GAS_NATURAL"


class TipoAutomovil(Enum):
    CIUDAD = "CIUDAD"
    SUBCOMPACTO = "SUBCOMPACTO"
    COMPACTO = "COMPACTO"
    FAMILIAR = "FAMILIAR"
    EJECUTIVO = "EJECUTIVO"
    SUV = "SUV"


class Color(Enum):
    BLANCO = "BLANCO"
    NEGRO = "NEGRO"
    ROJO = "ROJO"
    NARANJA = "NARANJA"
    AMARILLO = "AMARILLO"
    VERDE = "VERDE"
    AZUL = "AZUL"
    VIOLETA = "VIOLETA"


class Automovil:
    def __init__(self, marca: str, modelo: int, motor: float,
                 tipo_combustible: TipoCombustible,
                 tipo_automovil: TipoAutomovil, numero_puertas: int,
                 cantidad_asientos: int, velocidad_maxima: int, color: Color):
        self._marca = marca
        self._modelo = modelo
        self._motor = motor
        self._tipo_combustible = tipo_combustible
        self._tipo_automovil = tipo_automovil
        self._numero_puertas = numero_puertas
        self._cantidad_asientos = cantidad_asientos
        self._velocidad_maxima = velocidad_maxima
        self._color = color
        self._velocidad_actual = 0

    def get_marca(self) -> str:
        return self._marca

    def get_modelo(self) -> int:
        return self._modelo

    def get_motor(self) -> float:
        return self._motor

    def get_tipo_combustible(self) -> TipoCombustible:
        return self._tipo_combustible

    def get_tipo_automovil(self) -> TipoAutomovil:
        return self._tipo_automovil

    def get_numero_puertas(self) -> int:
        return self._numero_puertas
```

```
def get_cantidad_asientos(self) -> int:
    return self._cantidad_asientos

def get_velocidad_maxima(self) -> int:
    return self._velocidad_maxima

def get_color(self) -> Color:
    return self._color

def get_velocidad_actual(self) -> int:
    return self._velocidad_actual

def set_marca(self, marca: str) -> None:
    self._marca = marca

def set_modelo(self, modelo: int) -> None:
    self._modelo = modelo

def set_motor(self, motor: float) -> None:
    self._motor = motor

def set_tipo_combustible(self, tipo_combustible: TipoCombustible) -> None:
    self._tipo_combustible = tipo_combustible

def set_tipo_automovil(self, tipo_automovil: TipoAutomovil) -> None:
    self._tipo_automovil = tipo_automovil

def set_numero_puertas(self, numero_puertas: int) -> None:
    self._numero_puertas = numero_puertas

def set_cantidad_asientos(self, cantidad_asientos: int) -> None:
    self._cantidad_asientos = cantidad_asientos

def set_velocidad_maxima(self, velocidad_maxima: int) -> None:
    self._velocidad_maxima = velocidad_maxima

def set_color(self, color: Color) -> None:
    self._color = color

def set_velocidad_actual(self, velocidad_actual: int) -> None:
    self._velocidad_actual = velocidad_actual

def acelerar(self, incremento: int) -> None:
    if self._velocidad_actual + incremento <= self._velocidad_maxima:
        self._velocidad_actual += incremento
    else:
        print("No se puede incrementar a una velocidad superior "
              "a la máxima del automóvil.")

def desacelerar(self, decremento: int) -> None:
    if self._velocidad_actual - decremento >= 0:
        self._velocidad_actual -= decremento
    else:
        print("No se puede decrementar a una velocidad negativa.")

def frenar(self) -> None:
    self._velocidad_actual = 0

def calcular_tiempo_llegada(self, distancia: float) -> float:
    if self._velocidad_actual == 0:
        print("El automóvil está detenido: no se puede calcular el tiempo.")
        return float("inf")
    return distancia / self._velocidad_actual
```

```
    def imprimir(self) -> None:
        print(f"Marca = {self._marca}")
        print(f"Modelo = {self._modelo}")
        print(f"Motor = {self._motor} L")
        print(f"Tipo de combustible = {self._tipo_combustible.value}")
        print(f"Tipo de automóvil = {self._tipo_automovil.value}")
        print(f"Número de puertas = {self._numero_puertas}")
        print(f"Cantidad de asientos = {self._cantidad_asientos}")
        print(f"Velocidad máxima = {self._velocidad_maxima} km/h")
        print(f"Color = {self._color.value}")
        print(f"Velocidad actual = {self._velocidad_actual} km/h")

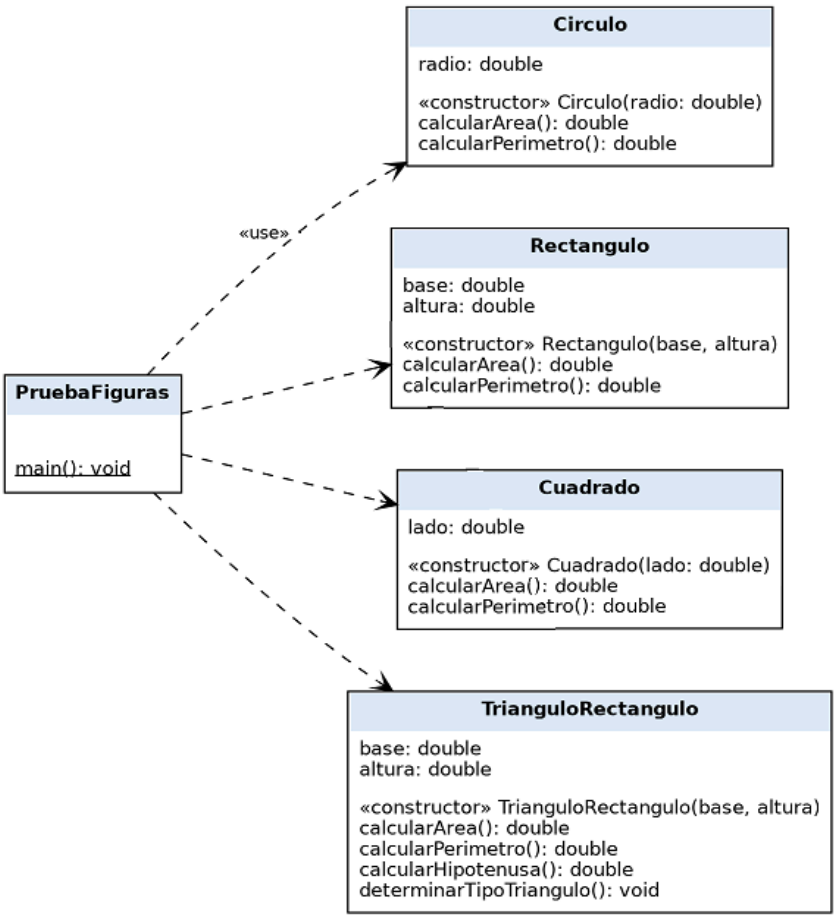

def main() -> None:
    auto1 = Automovil("Ford", 2018, 3.0, TipoCombustible.DIESEL,
                      TipoAutomovil.EJECUTIVO, 5, 6, 250, Color.NEGRO)
    auto1.imprimir()
    print()
    auto1.set_velocidad_actual(100)
    print(f"Velocidad actual = {auto1.get_velocidad_actual()}")
    auto1.acelerar(20)
    print(f"Velocidad actual = {auto1.get_velocidad_actual()}")
    auto1.desacelerar(50)
    print(f"Velocidad actual = {auto1.get_velocidad_actual()}")
    print(f"Tiempo estimado para 140 km = "
          f"{auto1.calcular_tiempo_llegada(140):.2f} h")
    auto1.frenar()
    print(f"Velocidad actual = {auto1.get_velocidad_actual()}")
    auto1.desacelerar(20)


if __name__ == "__main__":
    main()
```

Ejercicio Propuesto 2.4

Se requiere un programa que modele varias figuras geométricas: el círculo, el rectángulo, el cuadrado y el triángulo rectángulo.

Diagrama de clases



Código fuente (Python)

```
import math


class Circulo:
    def __init__(self, radio: float):
        self.radio = radio

    def calcular_area(self) -> float:
        return math.pi * self.radio ** 2

    def calcular_perimetro(self) -> float:
        return 2 * math.pi * self.radio


class Rectangulo:
    def __init__(self, base: float, altura: float):
        self.base = base
        self.altura = altura

    def calcular_area(self) -> float:
        return self.base * self.altura
```

```
    def calcular_perimetro(self) -> float:
        return 2 * self.base + 2 * self.altura


class Cuadrado:
    def __init__(self, lado: float):
        self.lado = lado

    def calcular_area(self) -> float:
        return self.lado ** 2

    def calcular_perimetro(self) -> float:
        return 4 * self.lado


class TrianguloRectangulo:
    def __init__(self, base: float, altura: float):
        self.base = base
        self.altura = altura

    def calcular_area(self) -> float:
        return self.base * self.altura / 2

    def calcular_hipotenusa(self) -> float:
        return math.sqrt(self.base ** 2 + self.altura ** 2)

    def calcular_perimetro(self) -> float:
        return self.base + self.altura + self.calcular_hipotenusa()

    def determinar_tipo_triangulo(self) -> None:
        a, b = self.base, self.altura
        c = self.calcular_hipotenusa()
        if math.isclose(a, b) and math.isclose(b, c):
            print("Es un triángulo equilátero")
        elif not (math.isclose(a, b) or math.isclose(a, c) or math.isclose(b, c)):
            print("Es un triángulo escaleno")
        else:
            print("Es un triángulo isósceles")


class PruebaFiguras:
    @staticmethod
    def main() -> None:
        figura1 = Circulo(2)
        figura2 = Rectangulo(1, 2)
        figura3 = Cuadrado(3)
        figura4 = TrianguloRectangulo(3, 5)

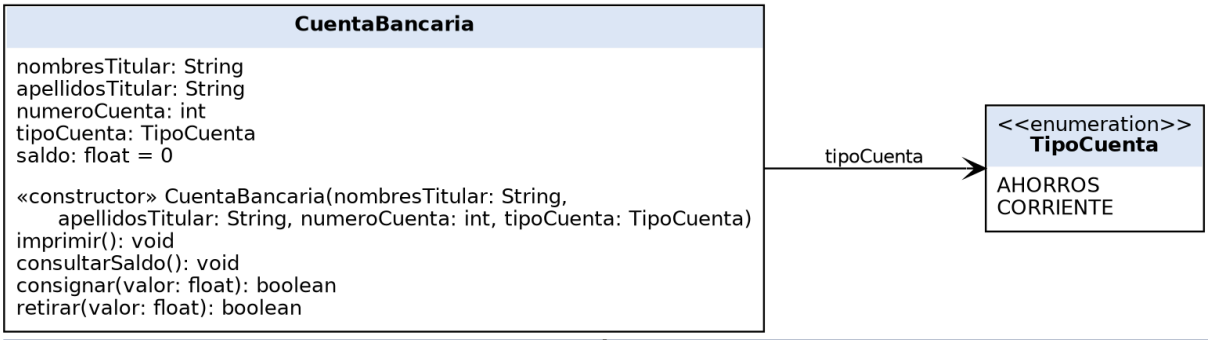
        print(f"El área del círculo es = {figura1.calcular_area()}")
        print(f"El perímetro del círculo es = {figura1.calcular_perimetro()}")
        print()
        print(f"El área del rectángulo es = {figura2.calcular_area()}")
        print(f"El perímetro del rectángulo es = {figura2.calcular_perimetro()}")
        print()
        print(f"El área del cuadrado es = {figura3.calcular_area()}")
        print(f"El perímetro del cuadrado es = {figura3.calcular_perimetro()}")
        print()
        print(f"El área del triángulo es = {figura4.calcular_area()}")
        print(f"El perímetro del triángulo es = {figura4.calcular_perimetro()}")
        print(f"La hipotenusa del triángulo es = {figura4.calcular_hipotenusa()}")
        figura4.determinar_tipo_triangulo()


if __name__ == "__main__":
    PruebaFiguras.main()
```

Ejercicio Propuesto 2.5

Se requiere un programa que modele una cuenta bancaria que posee los siguientes atributos: nombres del titular, apellidos del titular, número de la cuenta bancaria, tipo de cuenta (ahorros o corriente) y saldo de la cuenta.

Diagrama de clases



Código fuente (Python)

```
from enum import Enum


class TipoCuenta(Enum):
    AHORROS = "AHORROS"
    CORRIENTE = "CORRIENTE"


class CuentaBancaria:
    def __init__(self, nombres_titular: str, apellidos_titular: str,
                 numero_cuenta: int, tipo_cuenta: TipoCuenta):
        self.nombres_titular = nombres_titular
        self.apellidos_titular = apellidos_titular
        self.numero_cuenta = numero_cuenta
        self.tipo_cuenta = tipo_cuenta
        self.saldo = 0.0

    def imprimir(self) -> None:
        print(f"Nombres del titular = {self.nombres_titular}")
        print(f"Apellidos del titular = {self.apellidos_titular}")
        print(f"Número de cuenta = {self.numero_cuenta}")
        print(f"Tipo de cuenta = {self.tipo_cuenta.value}")
        print(f"Saldo = ${self.saldo:,.2f}")

    def consultar_saldo(self) -> None:
        print(f"El saldo actual es = ${self.saldo:,.2f}")

    def consignar(self, valor: float) -> bool:
        if valor > 0:
            self.saldo += valor
            print(f"Se ha consignado ${valor:,.2f} en la cuenta. "
                  f"El nuevo saldo es ${self.saldo:,.2f}")
            return True
        print("El valor a consignar debe ser mayor que cero.")
        return False

    def retirar(self, valor: float) -> bool:
        if 0 < valor <= self.saldo:
            self.saldo -= valor
            print(f"Se ha retirado ${valor:,.2f} de la cuenta. "
                  f"El nuevo saldo es ${self.saldo:,.2f}")
```

```
            return True
        print("El valor a retirar debe ser mayor que cero y no puede "
              "superar el saldo actual.")
        return False


def main() -> None:
    cuenta = CuentaBancaria("Pedro", "Pérez", 123456789, TipoCuenta.AHORROS)
    cuenta.imprimir()
    print()
    cuenta.consignar(200000)
    cuenta.consignar(300000)
    cuenta.retirar(400000)
    cuenta.retirar(400000)
    cuenta.consultar_saldo()


if __name__ == "__main__":
    main()
```

Repositorio en GitHub

https://github.com/JuanMFer24/actividadpoo1